

Tool-Calling Domain Specific Logic Document

Version 2.0 · March 24, 2026

This document specifies the domain-specific logic governing CoreThink's tool-calling execution engine. It defines, in formal terms, each sub-process that comprises the engine: how inputs are decomposed, how the scratchpad is initialised, how tasks are ordered, how prerequisites are checked, and how functions and parameters are matched and executed. Each sub-process is specified independently and is fully self-contained.

The five sub-processes described herein correspond directly to the phases illustrated in the CoreThink process flow diagram and together constitute the complete per-turn execution kernel.

Contents

- 1 Primitive Domains and Notation
- 2 Sub-process 1 — Decompose Input
- 3 Sub-process 2 — Initialise Scratchpad
- 4 Sub-process 3 — Get Order for Unit Tasks
- 5 Sub-process 4 — Check Prerequisites
- 6 Sub-process 5 — Match Functions and Parameters
- 7 Cross-Process Invariants

1 Primitive Domains and Notation

The following base sets and notation are used uniformly across all sub-processes in this document.

1.1 Base Sets

Identifier	Description
$V = \text{vocabulary}$	Universe of all possible values (strings, booleans, structured objects, etc.) e.g. 1, 2, 3, $\emptyset$, "x", True <i>nested objects of values</i>
K	Universe of all key names (identifiers); unified namespace for config keys, prerequisite keys, and parameter names
T <i>Sig</i>	Universe of all unit task descriptors
P	Universe of prerequisite keys; $P \subseteq K$ <i>why "unified" namespace why not hierarchical namespaces</i>
$F = \{D, F(V)\}$	Universe of callable function identifiers
$M = \{K_1: V_1, K_2: V_2\}$	Universe of parameter names; $M \subseteq K$
I	Universe of raw multi-turn input objects
C	Universe of initial configuration objects
$D = \text{python pkgs? or module?}$	Universe of tool and function registry objects
Sig	Universe of function signatures (name, parameter list, return type)

1.2 Notation

Identifier	Description
$\emptyset$	The empty set / empty partial function
$A \rightarrow B$	Total function from set A to set B (mapping?)
$A \rightharpoonup B$	Partial function from set A to set B
$\perp$	Undefined / absent value (bottom)
$x \mapsto v$ $k \mapsto v$	Binding of key x to value v assignment of v to key x why not $k \Leftarrow v$ arrow in other languages points in direction of operations and information flow
$S \cup \{x \mapsto v\}$ dict, update ($\{k, v\}$)	Scratchpad S extended with binding $x \mapsto v$; overwrites if x already present
$\text{dom}(f)$	Domain of partial function f (set of keys for which f is defined)
$\forall x \in A: P$	For all x in A, property P holds
$\exists x \in A: P$	There exists x in A such that P holds
$P \wedge Q$	Logical conjunction
$\neg P$	Logical negation
$\Rightarrow$	Logical implication
$ L $	Cardinality of list or set L

2 Sub-process 1 — Decompose Input

Receives the three raw inputs to the engine and separates them into the typed structures consumed by subsequent sub-processes. This is the entry point of the entire execution pipeline.

2.1 Inputs

Identifier	Description
$\iota \in \mathbb{I}$	The raw multi-turn input object
$\gamma \in \mathbb{C}$	The initial configuration object
$\Omega \in \mathbb{D}$	The tool and function registry

2.2 Procedure

```
DECOMPOSE( $\iota$ ,  $\gamma$ ,  $\Omega$ ):  
  
  turns     $\leftarrow$  Turns( $\iota$ )          : List( $\mathbb{T}$ )    – ordered sequence of unit tasks  
  derived from  $\iota$   
  config     $\leftarrow$  Extract( $\gamma$ )      :  $\mathbb{K} \rightarrow \mathbb{V}$     – key-value pairs extracted from  
  initial config  
  tools      $\leftarrow$  Register( $\Omega$ )      :  $\mathbb{F} \rightarrow \text{Sig}$   – map from function identifiers to  
  signatures  
  
  return  $\langle$  turns, config, tools  $\rangle$ 
```

2.3 Outputs

Identifier	Description
turns : List($\mathbb{T}$)	Raw (unordered) list of unit tasks; passed to GET-ORDER in Sub-process 3
config : $\mathbb{K} \rightarrow \mathbb{V}$	Initial scratchpad seed; written to S in Sub-process 2
tools : $\mathbb{F} \rightarrow \text{Sig}$	Available functions; consulted by MATCH-FUNC throughout Sub-process 5

2.4 Post-conditions

POST-1: $\text{dom}(\text{config}) \subseteq \mathbb{K}$ identifiers	– all config keys are valid
POST-2: $\text{dom}(\text{tools}) \subseteq \mathbb{F}$ identifiers	– all registered functions are valid
POST-3: $\text{turns} = \emptyset \vee \text{turns} \neq \emptyset$ terminates immediately	– empty input is valid; engine

3 Sub-process 2 — Initialise Scratchpad

Establishes the scratchpad S as a live, mutable key-value store seeded from the config output of DECOMPOSE. S persists and grows monotonically for the entire lifetime of the engine run.

3.1 Input

Identifier	Description
$\text{config} : \mathbb{K} \rightarrow \mathbb{V}$	Output of Extract(γ) from DECOMPOSE (Sub-process 1)

*config } state (stack) or memory
scratchpad }*

3.2 Procedure

INIT-SCRATCHPAD(config):

$S \leftarrow \emptyset$
function

- initialise scratchpad as empty partial

for each $(k, v) \in \text{config}$ do
 $S \leftarrow S \cup \{k \mapsto v\}$
end for

dict.update()

- write each config key-value pair into S

return S

*Config is just initialization
of state, so,*

3.3 Output

Identifier	Description
$S : \mathbb{K} \rightarrow \mathbb{V}$	Initialised scratchpad, seeded with all config key-value pairs. Shared globally across Sub-processes 4, 5, and 6.

3.4 Post-conditions

POST-1: $\text{dom}(S) = \text{dom}(\text{config})$
POST-2: $\forall k \in \text{dom}(\text{config}) : S(k) = \text{config}(k)$

*all variables
must be defined
in config? before
implementing the
algorithm or running
it?*

3.5 Invariant Established

INV-S (Monotonicity): from this point forward,
 $\forall$ states S_i, S_j where $i < j$: $\text{dom}(S_i) \subseteq \text{dom}(S_j)$

Keys are never deleted from S. Values may be overwritten, but the domain of S is strictly non-decreasing for the lifetime of the engine.

4 Sub-process 3 — Get Order for Unit Tasks

Takes the flat list of unit tasks from DECOMPOSE and produces a dependency-respecting execution order, represented as a tree where leaves (dependency-free tasks) execute first and the root executes last.

4.1 Input

Identifier	Description
turns : List(T)	Unordered list of unit tasks from DECOMPOSE (Sub-process 1)

4.2 Procedure

GET-ORDER(turns):	
G ← BUILD-DEP-GRAPH(turns)	— construct dependency graph $G = (V, E)$ where $V = \text{turns}$ and $(t_i, t_j) \in E$ iff t_i must execute before t_j
if CYCLE-DETECT(G) = true then	
Report(CyclicDependency, G)	
halt	
end if	
T ← TOPOLOGICAL-SORT(G)	— produce linear order consistent with E
return T	

4.3 Output — Execution Tree

Identifier	Description
T : List(T)	Ordered list of unit tasks, leaves first, root last. Passed to the main execution loop as the initial task queue.

The ordering is best understood as a dependency tree:

5 Sub-process 4 — Check Prerequisites

Evaluates all prerequisites for a single unit task t against the current scratchpad S . Returns either a green-light (Ready) or a single blocking task that must be resolved before t can proceed. This sub-process is fully self-contained and makes no assumptions about the task queue.

5.1 Inputs

Identifier	Description
$t : T$	The unit task whose prerequisites are to be checked
$s : \mathbb{K} \rightarrow \mathbb{V}$	The current scratchpad state

5.2 Procedure

```
CHECK-PREREQUISITES( $t$ ,  $S$ ):  
  
   $P \leftarrow \text{GET-PREREQS}(t)$            — retrieve ordered prerequisite key set for  
   $t$                                     $P \subseteq \mathbb{P}$   
  
  if  $P = \emptyset$  then  
    return  $\langle \text{Ready}, S \rangle$            — no prerequisites; task is immediately  
  ready                               ready  
  end if  
  
  for each  $p \in P$  do  
  
    result  $\leftarrow \text{CHECK}(p, S)$   
  
    case result of  
  
      | Met  $\rightarrow$                        — prerequisite satisfied; move to next  $p$   
        continue  
  
      |  $\neg \text{Met} \rightarrow$   
         $t' \leftarrow \text{TASK-TO-MEET}(p)$  — generate remediation task for  $p$   
        return  $\langle \text{Blocked}(t'), S \rangle$  — caller must execute  $t'$  before retrying  
  
      | Absent  $\rightarrow$   
         $t' \leftarrow \text{TASK-TO-CHECK}(p)$  — generate info-retrieval task for  $p$   
         $v \leftarrow \text{EXECUTE}(t')$        — retrieve value inline
```

```

    S ← S ∪ { p ↦ v }      - write result to scratchpad
    continue                - re-enter loop; p now in dom(S)

end case

end for

return < Ready, S >        - all prerequisites satisfied

```

5.3 CHECK Function

```

CHECK : P × (K → V) → { Met, ¬Met, Absent }

CHECK(p, S) =
  | Met      if p ∈ dom(S) ∧ Satisfied(p, S(p))
  | ¬Met     if p ∈ dom(S) ∧ ¬Satisfied(p, S(p))
  | Absent   if p ∉ dom(S)

```

where $Satisfied(p, v)$ is a domain-specific predicate determining whether value v constitutes a satisfactory resolution for prerequisite p .

5.4 Outputs

Identifier	Description
< Ready , S' >	All prerequisites in P are Met. Task t may proceed to Sub-process 5.
< Blocked (t'), S >	Prerequisite p was ¬Met. Task t' must be executed and t retried. S is unchanged.

5.5 Output Tree

```

CHECK-PREREQUISITES(t, S)
|
|—— P = ∅ —————→ < Ready, S >
|
|—— ∀ p ∈ P: Met —————→ < Ready, S >
|
|—— ∃ p ∈ P: ¬Met —————→ < Blocked(t'), S >
|                               where t' = TASK-TO-MEET(p)
|
|—— ∃ p ∈ P: Absent
|
|   |—— EXECUTE(t') → v —→ S' = S ∪ { p ↦ v }, continue loop

```

5.6 Invariants

INV-1 (Scratchpad monotonicity):

$S' \supseteq S$ — the Absent branch only ever adds to S , never removes

INV-2 (Absent terminates):

After the Absent branch, $p \in \text{dom}(S')$

$\therefore \text{CHECK}(p, S') \in \{ \text{Met}, \neg \text{Met} \}$ on any subsequent call

$\therefore$ the Absent branch fires at most once per prerequisite key p

INV-3 (Single block per call):

At most one $\text{Blocked}(t')$ is returned per invocation.

The caller is responsible for resolving t' and retrying.

generative

→ what do you do when
two dependencies use the
same IK for different
forms of some
thing — e.g. DI
water, tap water,
filtered water?

global scratchpad
should be name-spaced
like module & package
hierarchies

6 Sub-process 5 — Match Functions and Parameters

Entered after all prerequisites for task t are satisfied. Identifies the function to invoke, resolves all required parameters from the turn context and scratchpad, and either executes the function or returns a typed failure. This sub-process is fully self-contained and makes no assumptions about the task queue.

6.1 Inputs

Identifier	Description
$t : T$	The current unit task (all prerequisites satisfied)
$S : K \rightarrow V$	The current scratchpad state
$\tau : M \rightarrow V$	The current turn context (read-only; takes priority over S)
$\Omega : F \rightarrow \text{Sig}$	The tool and function registry

6.2 Procedure

```
MATCH-FUNCS-AND-PARAMS( $t, S, \tau, \Omega$ ):

  // — Stage 1: Function Matching —————

   $f \leftarrow \text{MATCH-FUNC}(t, \Omega)$            — find best matching function for  $t$  in  $\Omega$ 

  if  $f = \perp$  then
    Report(MissingFunction,  $t$ )
    return < Failure(MissingFunction),  $S$  >
  end if

  // — Stage 2: Parameter Resolution —————

   $M \leftarrow \text{PARAMS}(f)$                  — retrieve ordered set of required
parameter names                                $M \subseteq M$ 

   $B \leftarrow \emptyset$                    — initialise binding accumulator  $B : M \rightarrow V$ 

  for each  $m \in M$  do

     $v \leftarrow \text{LOOKUP}(m, \tau, S)$        — consult turn context  $\tau$  first, then
scratchpad  $S$ 
```

```

case v of

| v ≠ ⊥ →
    B ← B ∪ { m ↦ v }          - parameter found; bind and continue

| v = ⊥ →
    t' ← TASK-TO-GET(m)         - generate retrieval task for m
    v' ← EXECUTE(t')           - attempt to retrieve value
    S ← S ∪ { m ↦ v' }         - write to scratchpad regardless of outcome

    if v' = ⊥ then
        Report(MissingParam, m)
        return < Failure(MissingParam(m)), S >
    else
        B ← B ∪ { m ↦ v' }     - bind retrieved value and continue
    end if

end case

end for

// — Stage 3: Execution —————

// Precondition: dom(B) = M   (all parameters fully bound)
EXECUTE(f, B)

return < Success, S >

```

6.3 LOOKUP Function

```

LOOKUP : M × (M → V) × (K → V) → V ∪ {⊥}

LOOKUP(m, τ, S) =
| τ(m)    if m ∈ dom(τ)          - turn context takes priority
| S(m)    if m ∉ dom(τ) ∧ m ∈ dom(S) - fall back to scratchpad
| ⊥       otherwise             - parameter not available

```

6.4 Outputs

Identifier	Description
< Success , S' >	f was matched and executed with all parameters fully bound
< Failure(MissingFunction) , S >	No function in Ω matched t. Task abandoned. S unchanged.

Identifier	Description
$\langle \text{Failure}(\text{MissingParam}(m)), S' \rangle$	Parameter m could not be resolved. Task abandoned. S' includes the $\perp$ write for m .

6.5 Output Tree

```

MATCH-FUNCS-AND-PARAMS( $t, S, \tau, \Omega$ )
  |
  |— MATCH-FUNC( $t, \Omega$ ) =  $\perp$  —————→ Failure(MissingFunction)
  |
  |— MATCH-FUNC( $t, \Omega$ ) =  $f$ 
    |
    |—  $\forall m \in M: \text{LOOKUP}(m, \tau, S) \neq \perp$  —→ EXECUTE( $f, B$ ) →
Success
    |
    |—  $\exists m \in M: \text{LOOKUP}(m, \tau, S) = \perp$ 
      |
      |— EXECUTE( $t'$ ) =  $v' \neq \perp$  —→  $B \leftarrow B \cup \{m \mapsto v'\},$ 
continue
      |
      |— EXECUTE( $t'$ ) =  $\perp$  —————→
Failure(MissingParam( $m$ ))

```

6.6 Invariants

INV-1 (Execution guard):
 EXECUTE(f, B) is called if and only if $\text{dom}(B) = M$
 Partial execution is never attempted.

INV-2 (Scratchpad write-on-retrieve):
 $S \leftarrow S \cup \{m \mapsto v'\}$ is applied even when $v' = \perp$
 $\therefore \text{LOOKUP}(m, \tau, S) \neq \perp$ on any subsequent call for the same m
 $\therefore$ the retrieval branch fires at most once per parameter name m

INV-3 (Lookup priority):
 τ is always consulted before S
 $\therefore$ turn-local values are never silently overridden by stale scratchpad entries

INV-4 (Failure is non-fatal):
 Both Failure modes abandon only task t .
 The caller continues with the next task in T .

7 Cross-Process Invariants

The following invariants hold globally across all sub-processes for the lifetime of a single engine run.

7.1 Scratchpad Monotonicity (INV-S)

$\forall$ states S_i, S_j observed at steps $i < j$:
 $\text{dom}(S_i) \subseteq \text{dom}(S_j)$

Keys are never removed from S . This is established in Sub-process 2 and upheld by every subsequent write in Sub-processes 4 and 5.

Memory leak?
what if output is a
long sequence such as
an n-gram tokenizer?
no concept of
a python generator?

7.2 Task Queue Well-Foundedness (INV-T)

The engine terminates if and only if:

- (1) The set of unit tasks reachable via Turns, TASK-TO-MEET, TASK-TO-CHECK, and TASK-TO-GET is finite.
- (2) The prerequisite dependency graph G is acyclic (DAG).
- (3) The parameter dependency graph is acyclic.

Violation of (2) is caught and halted in Sub-process 3 (GET-ORDER).
Violations of (1) and (3) are the caller's responsibility.

7.3 Execution Precondition (INV-E)

$\text{EXECUTE}(f, B)$ is called if and only if $\text{dom}(B) = \text{Params}(f)$.

This is guaranteed by Sub-process 5 (MATCH-FUNCS-AND-PARAMS).
No other sub-process calls $\text{EXECUTE}(f, B)$ directly.

7.4 Non-Fatal Errors (INV-R)

$\text{Report}(E, x)$ does not modify S , T , or τ .
 $\text{Report}(E, x)$ causes abandonment of the current task t only.
Execution continues with the next task in T after any Report call.

7.5 Data Flow Summary

